

Date: 04/08/2020

Title :- Implementation of Uninformed Search Algorithms (Breadth First Search and Depth First Search).

AIM:-

Implement Breadth First Search (BFS) and Depth first search (DFS) algorithms for solving route-finding problems. Compare their performance based on path-length, nodes explored and execution time.

Theory:-

1) What is a Search Problem in Artificial Intelligence? Explain its components.

→ A search problem in Artificial Intelligence is a problem in which an agent searches through different possible states to find a path from an initial state to a desired goal state.

The main components of a search problem are:-

1) Initial state → The starting point of the problem.

2) Goal state → The desired destination or solution.

3) Actions → The possible operations that can be performed from a state.

4) Transition model:- Describes the state obtained after performing an action.



Date : _____

5) Path cost - The cost associated with reaching a particular state.

2) What ~~are~~ are Uninformed Search Algorithms? Explain BFS and DFS.

→ Uninformed search algorithms, also called blind search algorithms, do not use additional information or heuristics about the goal. They search the problem space using only the information provided in the problem.

a) BFS (Breadth First Search)

BFS explores nodes level by level. It uses a queue, following the First-In-First-out (FIFO) principle.

Working →

- Start from the initial node.
- Insert it into the queue.
- Remove a node from the front of the queue.
- Mark it as visited.
- Add its unvisited neighbours to the queue.
- Continue until the goal node is found or the queue becomes empty.

- Advantages -

- Find the shortest path in an unweighted graph.
- Complete if a solution exists
- Simple to implement.

- Disadvantages -

- Requires more memory
- Can explore many unnecessary nodes.

b) DFS (Depth First Search)

DFS explores ~~within~~ one branch as deeply as possible before backtracking. Your implementation uses a stack, following the Last-In-First-out (LIFO) principle.

Working:-

- Start from initial node.
- Push it into the stack.
- Remove a node from the top of the stack.
- Mark it as visited.
- Push its unvisited neighbours onto the stack.
- Continue until the goal is found or the stack becomes empty.

- Advantages:-

- Requires less memory than BFS
- Can reach a deep goal quickly
- Simple to implement.

- Disadvantages:-

- Does not always find the lowest path.
- May explore a large or irrelevant branch first.

Q3) Write the algorithm for BFS & DFS.

→ BFS Algorithm

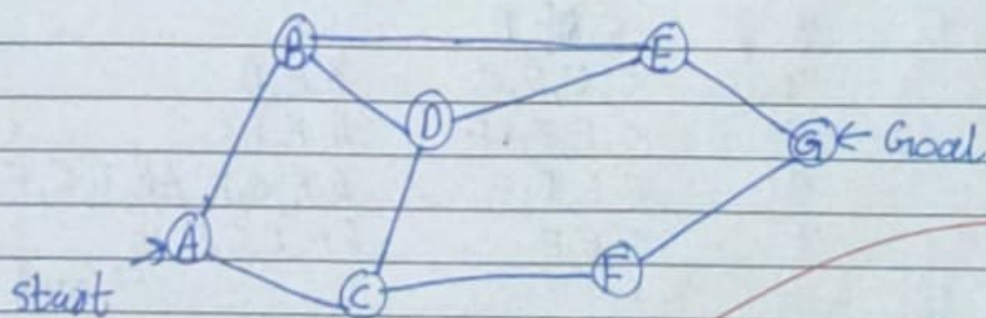
- 1) Start with the initial node
- 2) Create an empty visited list and a queue.
- 3) Insert the starting node into the queue.
- 4) While the queue is not empty:-
 - Remove the front node.
 - If it is the goal node, stop.
 - If it is already visited, skip it.
 - Add it to the visited list
 - Insert its unvisited neighbours into the queue.
- 5) If the queue becomes empty, the goal cannot be reached.

→ DFS Algorithm

- 1) Start with the initial node
- 2) Create an empty visited list and a stack.
- 3) Push the ~~starting~~ starting node onto the stack
- 4) While the stack is not empty:
 - Remove the top node.
 - If it is already visited, skip it.
 - If it is the goal node, stop.
 - Add it to the visited list.
 - Push its unvisited neighbours onto the stack in reverse order.

5) If the stack becomes empty, the goal cannot be reached.

Q4) Solve the following problem. Show the step by step working (use both BFS & DFS)





Date : _____

→ BFS Working:-

Given, Starting = A, Goal = G

Step	Queue	Visited
1	A	-
2	B, C	A
3	C, D, E	A, B
4	D, E, F	A, B, C
5	E, F	A, B, C, D
6	F, G	A, B, C, D, E
7	G, G	A, B, C, D, E, F
8	-	A, B, C, D, E, F, G

∴ BFS Path → A → B → C → D → E → F → G

→ DFS Working:- Given, Start = A, Goal = G.

Step Stack Visited

1	A	-
2	C, B	A
3	C, D, E	A, B
4	C, E, F, C	A, B, D
5	C, E, F, D	A, B, D, C
6	C, E, F, G	A, B, D, C, F
7	C, E, F	A, B, D, C, F, G ↳ Goal

∴ DFS Path → A → B → D → C → F → G



Date : _____

Q4) Compare their performance based on path-length, nodes explored and execution time.

→

Parameter	BFS	DFS
Data Structure	Queue	Stack
Searching method	Level by level	Depth wise
Path length	Generally shortest in an unweighted graph	Not necessarily shortest.
Memory Usage	Higher	Lower
Nodes Explored	may explore more nodes	may explore fewer or more depending on graph.
Completeness	Complete for finite branching graphs	Not always complete in infinite deep spaces.
Time Complexity	$O(V+E)$	$O(V+E)$
Space Complexity	$O(V)$	$O(V)$
Suitable for	Shortest route finding	Deep exploration.



Date : _____

Conclusion

BFS and DFS were implemented successfully for route finding. BFS explores nodes level by level and generally gives the shortest path in an unweighted graph, while DFS explores deeper branches first and uses less memory. The assignment demonstrates the difference between queue-based and stack-based search.